

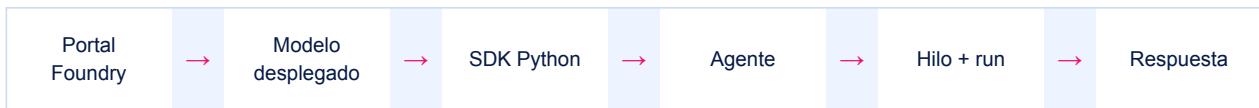
Sesión 1 · Tu primer agente

Construye, conversa y añade herramientas sin dar nada por supuesto.

01

Tu meta de aprendizaje

Al terminar podrás explicar la arquitectura mínima de un agente, conectarte a Foundry, crear una conversación y añadir herramientas.



- Producto mínimo: agente conversacional en español.
- Extensión 1: cálculo exacto con Code Interpreter.
- Extensión 2: función Python elegida por el agente.

1. Entiende las piezas antes de programar

Pieza	Qué representa	Pregunta de control
Modelo	El motor que genera la respuesta.	¿Qué deployment usaré?
Agente	Modelo + instrucciones + herramientas.	¿Qué rol y límites tiene?
Hilo	Historial de una conversación.	¿Qué debe recordar?
Run	Un turno de razonamiento y ejecución.	¿Terminó o falló?

2. Prepara Azure y el proyecto

- 1 Verifica herramientas
Python 3.9+ y Azure CLI deben responder desde la terminal.
- 2 Crea el proyecto
En el portal de Foundry crea un proyecto con nombre sin espacios ni acentos.
- 3 Despliega el modelo
Elige un modelo disponible, anota el nombre exacto del deployment y revisa la cuota.
- 4 Copia el endpoint
Usa el endpoint del proyecto; debe apuntar al proyecto, no solo al servicio de modelos.

CONTROL DE COSTOS

Revisa precio, cuota y región antes de desplegar. Al terminar, elimina los recursos que no conservarás.

3. Configura Python y las credenciales

```
python3 -m venv .env
source .env/bin/activate # Windows PowerShell: .env\Scripts\Activate.ps1
pip install -r requirements.txt
cp .env.example .env
az login
```

```
FOUNDRY_PROJECT_ENDPOINT=https://.../api/projects/tu-proyecto
FOUNDRY_MODEL_NAME=tu-deployment
```

VERSIÓN VERIFICADA

Este manual usa azure-ai-projects 2.4.0 (API v1). Instala `azure-ai-projects>=2.4.0,<2.5` y comprueba con `pip show azure-ai-projects`.

SEGURIDAD

Nunca subas `.env`. Añade `.env`, `.venv/` y archivos con secretos a `.gitignore`.

4. Crea y conversa con el agente

Guarda el siguiente programa como `agente.py`. Observa el flujo vigente: conectar → crear una versión del agente → crear conversación → solicitar una `Response`.

```
import os
from dotenv import load_dotenv
from azure.ai.projects import AIProjectClient
from azure.ai.projects.models import PromptAgentDefinition
from azure.identity import DefaultAzureCredential

load_dotenv()
project = AIProjectClient(
    endpoint=os.environ["FOUNDRY_PROJECT_ENDPOINT"],
    credential=DefaultAzureCredential(),
)
openai = project.get_openai_client()
agent = project.agents.create_version(
    agent_name="mi-primer-agente",
    definition=PromptAgentDefinition(
        model=os.environ["FOUNDRY_MODEL_NAME"],
        instructions="Responde en español, de forma breve y verificable.",
    ),
)
conversation = openai.conversations.create()
response = openai.responses.create(
    conversation=conversation.id,
    input="¿Qué puedes hacer por mí?",
    extra_body={"agent_reference": {
        "name": agent.name, "type": "agent_reference"
    }},
)
print(response.output_text)
```

5. Añade herramientas

Code Interpreter. En 2.x las herramientas se importan desde `azure.ai.projects.models` y se agregan a `PromptAgentDefinition`.

```
from azure.ai.projects.models import (
    PromptAgentDefinition, CodeInterpreterTool,
    AutoCodeInterpreterToolParam,
)

ci = CodeInterpreterTool(
    container=AutoCodeInterpreterToolParam(file_ids=[])
)
agent = project.agents.create_version(
    agent_name="agente-calculadora",
    definition=PromptAgentDefinition(
        model=os.environ["FOUNDRY_MODEL_NAME"],
        instructions="Usa código para calcular y explica el resultado.",
        tools=[ci],
    ),
)
```

Function calling. Declara un FunctionTool con JSON Schema. La primera Response puede devolver `function_call`; tu aplicación valida argumentos, ejecuta la función y envía `FunctionCallOutput` en una segunda Response.

```
from azure.ai.projects.models import FunctionTool

clima_tool = FunctionTool(
    name="obtener_clima",
    description="Obtiene el clima actual de una ciudad.",
    parameters={
        "type": "object",
        "properties": {"ciudad": {"type": "string"}},
        "required": ["ciudad"],
        "additionalProperties": False,
    },
    strict=True,
)
```

6. Prueba y demostración

1

Caso feliz

Haz una pregunta alineada con las instrucciones.

2

Herramienta

Usa una operación grande y verifica el cálculo fuera del agente.

3

Límite

Pregunta por una ciudad no incluida: debe reconocer que no tiene datos.

4

Evidencia

Guarda captura o salida de terminal y explica qué herramienta se activó.

DEFINICIÓN DE TERMINADO

El agente responde, un caso obliga a usar herramienta, un caso límite no inventa y el repositorio no contiene secretos.

7. Diagnóstico rápido

401 / token. Ejecuta `az login`; revisa `tenant` y permisos.

404 deployment. Copia el nombre exacto del deployment.

Endpoint incorrecto. Comprueba que sea el endpoint del proyecto.

`ModuleNotFoundError`. Activa `.venv` e instala `requirements.txt`.

429 / cuota. Espera, reduce concurrencia o elige un deployment con cuota.

8. Mini reto y reflexión

- Cambia el rol del agente por uno útil en tu contexto.
- Diseña una función con entradas claramente tipadas.
- Escribe tres pruebas: normal, ambigua y fuera de alcance.
- Explica en 60 segundos la diferencia entre modelo, agente, hilo y herramienta.